VIAM

Welcome!

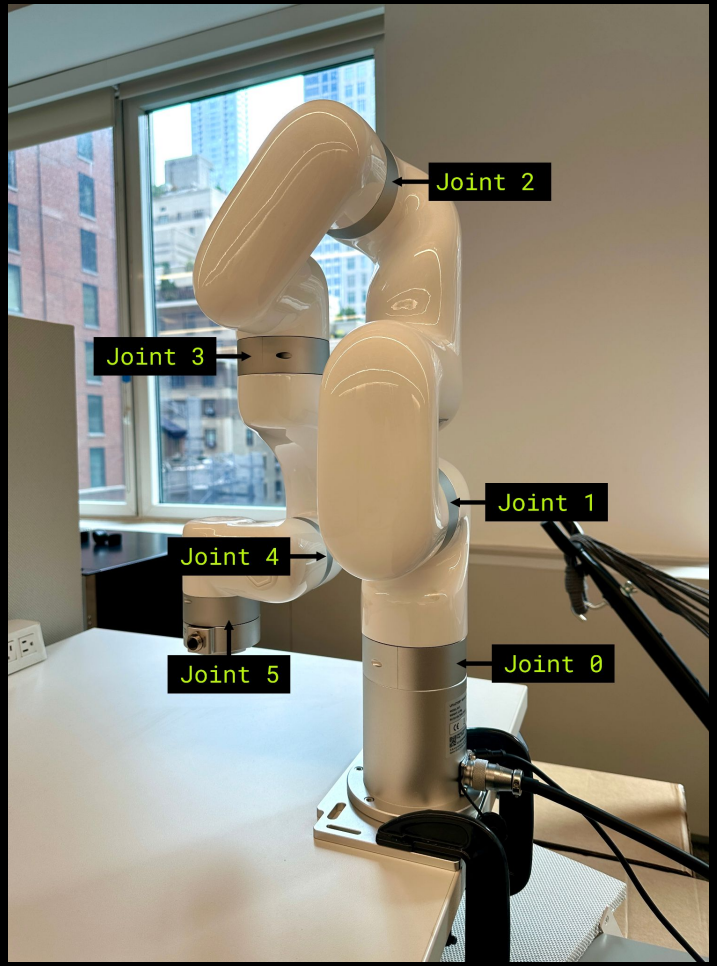
Schedule

- **9:00 AM:** Check-in and breakfast
- **9:30 AM:** Kickoff: Welcome & crash course
- **10:00 AM:** Team formation
- **10:15 AM:** Get started
- **1:00 PM:** Lunch
- **6:00 PM:** Dinner
- 9:00 PM: Hard close

Hardware

The arm

- `viam:ufactory:xArm6` component or
- `viam:ufactory:xArm850` component
- **Joint positions**: move each of the six joints
- **Jogging**: nudge a joint and watch the arm follow. A jog executes immediately on release.
- **End position**: X, Y, Z, plus the orientation vector
- Each control calls an arm API method: `MoveToJointPositions`, `MoveToPosition`, `GetJointPositions`, `IsMoving`



The gripper

- `viam:ufactory:gripper` component
- The gripper names the arm it's attached to.
- Test it with Open / Grab, and check `IsHoldingSomething` once it grabs a block

The camera

- `viam:camera:realsense` component
- Intel Realsense D435
- RGB
- Stereo depth

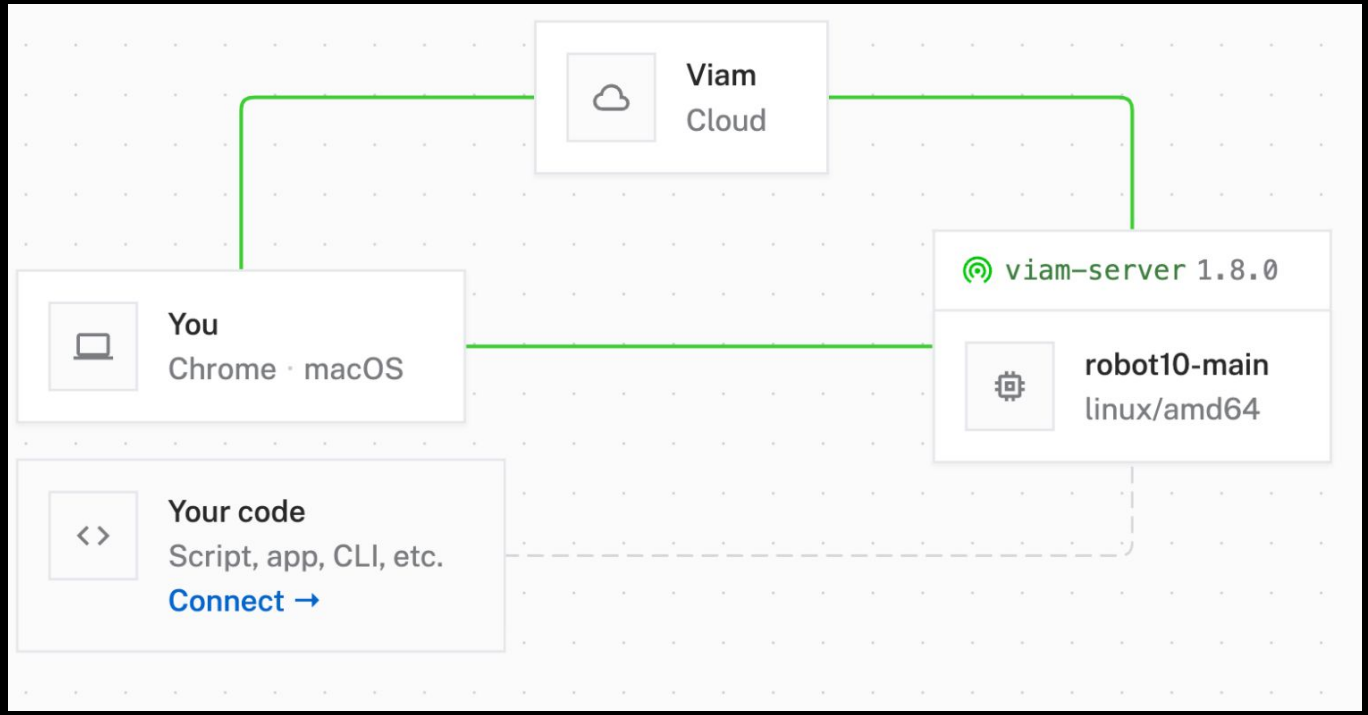


Before the arm moves

- Know where the E-stop is
  - An E-Stop is a physical emergency stop button that halts the arm immediately. Hit it if the arm is about to hit something/someone.
- Obstacles on your machine don't stop direct arm moves. More on obstacles later.



Viam system overview



Configuration & The Module Registry

Test / Control Cards

- You can interact with the arm, the gripper, camera feeds, and data from vision and other services
- Every component and service has test cards

Frames

- A frame is a coordinate system, an origin and x, y, and z axes, attached to a component. A frame is expressed in translation and orientation:
  - Translation: the offset from the parent, in millimeters
  - Orientation: the direction the component faces
- Every frame has a parent.
- A new frame's parent is the world frame by default.
- The 3D scene is empty if components don’t have frames.

The frames on your machine

Frame	Parent	Translation	Orientation
arm	world	{ "x": 0, "y": 0, "z": 0 }	{ "x": 0, "y": 0, "z": 1, "th": 0 }
gripper	arm	{ "x": 0, "y": 0, "z": 105 }	{ "x": 0, "y": 0, "z": 1, "th": 0 }
cam	arm	{ "x": 73, "y": -40, "z": 18 }	{ "x": 0, "y": 0, "z": 1, "th": 90 }

- The arm sits at the world origin
- The gripper and camera are parented to the arm, so they move with the arm
- The gripper's frame is 105 mm out from the end of the arm (probably)
- The above camera's frame is for RealSense cameras. It's good enough for most pick and place
- For very precise camera frame calibration, use the `viam:camera-calibration:handeye` service

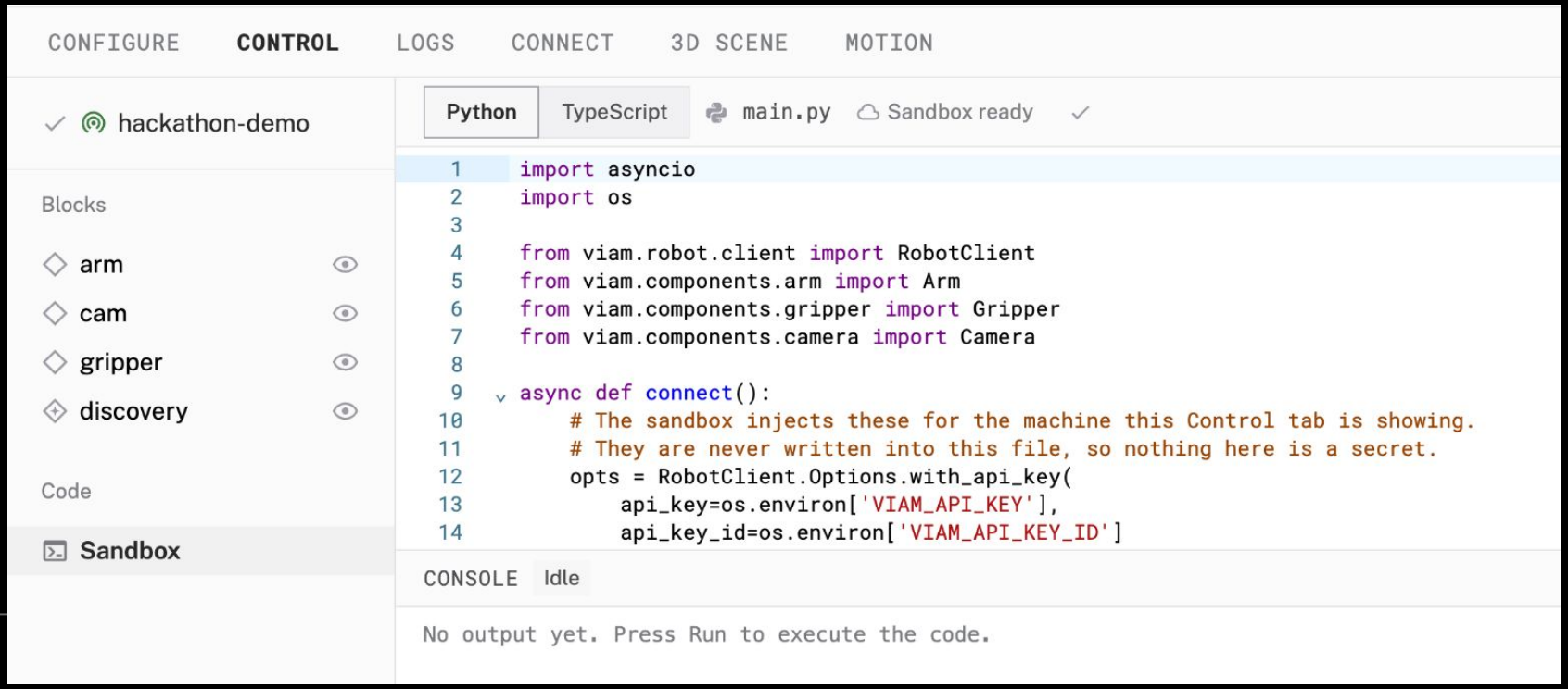
Your machine in 3D

- The 3D SCENE tab renders your machine's frame system
- Move the arm and the arm in the scene moves
- The gripper and camera move with it, because their frames are parented to the arm
- Your machine has `arm`, `gripper`, and `cam`. Every machine will be configured with these components, plus table and wall obstacles.

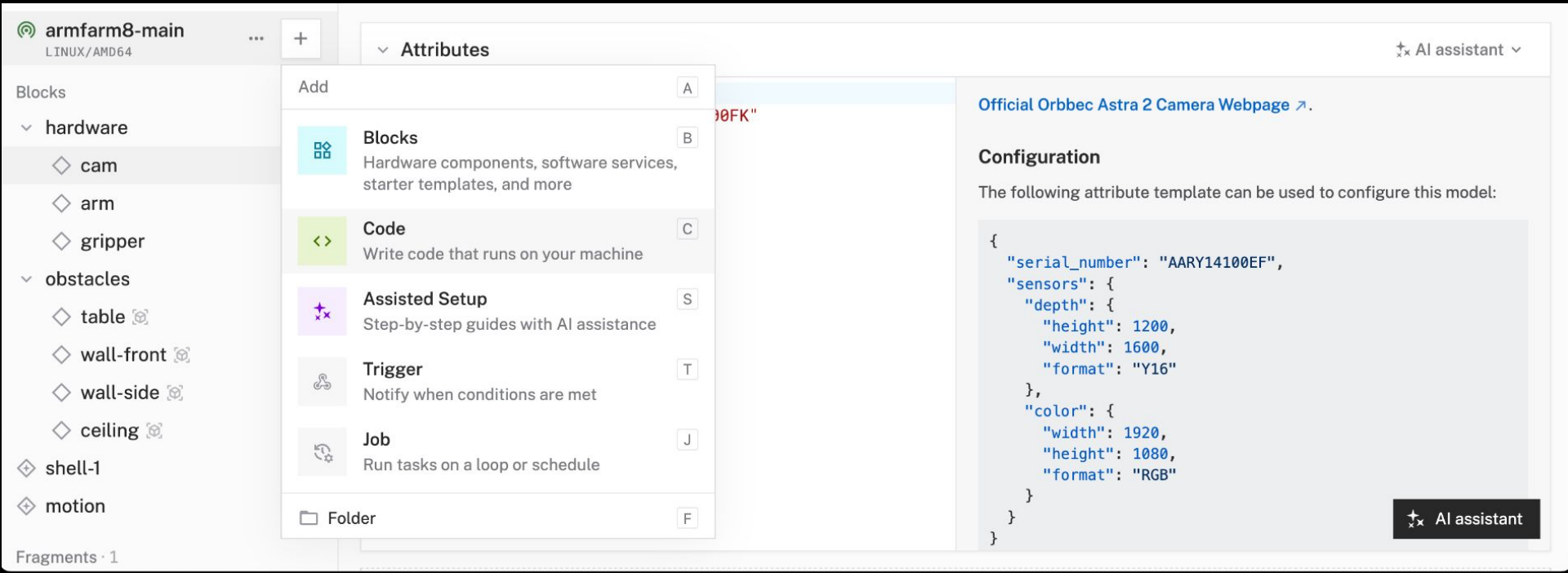
Writing control logic

Sandbox

- You can write test SDK code in the browser and execute it from the Sandbox environment in the CONTROL tab. You can prototype something quickly here.



Inline modules



Registry modules

WHAT IS VIAM?

TRY

SET UP A MACHINE

CONFIGURE HARDWARE

VIAM CLI

MANAGE DATA

TRAIN ML MODELS

COMPUTER VISION

BUILD AND DEPLOY MODULES

- Overview
- Check the registry first
- Anatomy of a module
- Write an inline module
- Write a driver module
- Write a logic module
- Deploy a module
- Update and manage modules

Build and deploy modules

Modules extend what your machine can do. They come in two varieties: driver modules that add support for new hardware, and logic modules that tie components together with decision-making code. You can develop modules in your own IDE or write them directly in the browser using inline modules.

If you have already configured components on your machine, each one works individually: you can test it from the Viam app, capture its data, and call its API from a script. The next step is making them work **together**. A camera detects an object, and a motor responds. A temperature sensor crosses a threshold, and a notification fires. A movement sensor reports position, and an arm adjusts.

To learn module development step by step, take the free Viam 102 course:

FREE COURSE

Viam 102: Module Development

Turn a control script into a production-grade module: lifecycle, resource management, and publishing to the registry.

120 minutes · No hardware required

Vision services

- A vision service turns camera images into detections your code can act on: bounding boxes, class labels, or 3D point cloud objects
- Recommendations:
  - Shape finder (`devrel:shape-finder:detector`) with detections to segments (`viam:vision:detections-to-segments`)
  - YOLOv8 (`viam-labs:vision:yolov8`): object detection or classification. Point its `model_location` at a stock model name, a local model file, or a Hugging Face repo id, which it downloads and runs locally
  - A pre-trained model: the built-in ML model vision service (`rdk:builtin:mlmodel`) running the `viam-labs/EfficientDet-COCO` object detector on the TFLite ML model service `viam:mlmodel-tflite:tflite_cpu`
  - Hand gesture recognition (`viam:hand-gesture-recognition:gestures`): recognizes 7 hand gestures as detections
  - AprilTag pose tracker (`viam:apriltag:pose_tracker`): tracks the pose of AprilTag markers

Plan motion around obstacles

- The built-in motion service plans a path to a destination and avoids collisions along the way
- Your machine has a table obstacle and a wall obstacle (`erh:vmodutils:obstacle`). The motion service plans around them. Direct arm moves (`MoveToPosition` and `MoveToJointPositions`) go straight through them.
- Obstacles help plan paths. Remember to use the E-stop to immediately stop a moving arm.

Capture data

- The data manager service records data from components and services, such as camera images and joint positions, and syncs it to the Viam app
- Query it, export it, or use it to train a machine learning model
- Useful for reviewing what the camera saw when a grasp went wrong, or collecting images to train a vision model

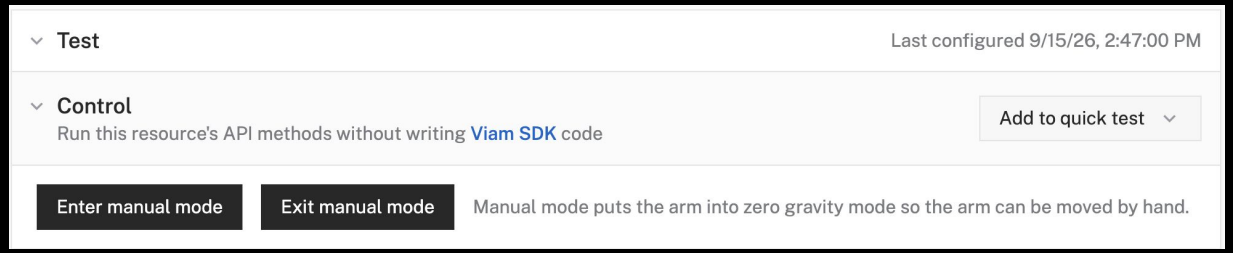
Mental Model

With the steps and states of your application in mind:

- What vision pipeline(s) do you need?
  - What modules already support the elements of this pipeline?
  - Are there any pipeline elements Claude and I need to write?
- What static poses do I need for my arm?
- What portions of my application require motion planning?
- Do I need to collect data?
  - If so, check the docs or ask a Viam team member about data capture and training
- For all of the above, what software modules do I need to write?
  - What interface does the rest of the system need to these modules?
  - You'll probably use the Generic API and implement the interface as DoCommands (ask us).

Tips and helpers

- Manual mode: switch the arm into manual mode from the arm Configure panel, then move it by hand to find a position



- Arm position saver (`erh:vmodutils:arm-position-saver`): records the arm's current pose, and can be used to move the arm back to it later
- Quick test tray: pin a control so it stays available as you move around the app
- Picture-in-picture: keep a camera feed in view while you move the arm

Using AI

- The AI assistant in the Viam app
- Your own AI agent: you ca AI assistant at Viam Docs or the agent-skills repository
- The Viam MCP server is available at https://app.viam.com/mcp
  - In Claude, go to Customize > Connectors > Add
  - In ChatGPT, go to Plugins > + Icon
    - You'll need to enable developer mode to be able to add custom plugins. You can do this by going to Settings > Security and login > Developer mode

Reference

Discovery services

- For RealSense cameras: `viam:realsense:discovery`
- For Orbbec cameras: `viam:orbbec:discovery`
- For webcams: `viam:find-webcams:webcam-discovery`
- Refresh it to find the connected camera(s)
- Add the camera you want

Tutorial

- Pick-and-place tutorial uses the same hardware setup

Resources & Getting Unstuck

- Ask a Viam staff!
- Viam docs
- Viam registry

Viam Crash Course

Share your build

- Excited about your build? Our video team's around if you want to talk about it or show it off.
- Tag us on socials @viamrobotics
- Share in the Robotics topic on Hackster.io https://www.hackster.io/robotics



Hackster.io

Find your machine

- Go to https://app.viam.com
- Organization > location > machine
- Find your machine in the Arm Farm location of the hackathons organization

VIAM

Happy Building!


VIAM